VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELGAUM, KARNATAKA

PROJECT REPORT ON

"DESIGN AND IMPLEMENTATION OF HAND GESTURE CONTROLLED NAVIGATION SYSTEM FOR HUMAN COMPUTER INTERACTION"

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

SUBMITTTED BY
2SD23CS002 ABHILASH PYATI
2SD23CS041 KARAN KUMBAR
2SD23CS061 NAVEEN GURIKAR
2SD23CS086 SAMARTH SANIKOP

UNDER THE GUIDANCE OF
DR. SANDHYA S V

DEPARTMENT OF COMPUTER SCIENCE AND ENGG
S.D.M. COLLEGE OF ENGINEERING AND TECHNOLOGY
DHARWAD, KARNATAKA, INDIA

ACADEMIC YEAR 2025–2026

S.D.M. COLLEGE OF ENGINEERING AND TECHNOLOGY,

DHARWAD–580002

CERTIFICATE

This is to certify that the project titled **DESIGN AND IMPLEMENTATION OF HAND GESTURE CONTROLLED NAVIGATION SYSTEM FOR HUMAN COMPUTER INTERACTION** is a Bonafide work carried out by **Abhilash Pyati (2SD23CS002), Karan Kumbar (2SD23CS041) , Naveen Gurikar (2SD23CS061) , Samarth Sanikop (2SD23CS086)** submitted in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING** of **S.D.M.COLLEGE OF ENGINEERING AND TECHNOLOGY, DHARWAD, KARNATAKA** (An autonomous institution affiliated to Visvesvaraya Technological University, Belgaum, Karnataka), during the year **2025–2026**.

It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The project has been approved, as it satisfies the academic requirements in respect of project report prescribed for the said degree.

Dr. Sandhya S V
Project Guide

Dr. U. P. Kulkarni
Head of Department

Name of Examiners

Signature with Date

1)

2)

DECLARATION

We hereby declare that the dissertation work titled **DESIGN AND IMPLEMENTATION OF HAND GESTURE CONTROLLED NAVIGATION SYSTEM FOR HUMAN COMPUTER INTERACTION**, has been carried out under the guidance of **Dr. Sandhya S V, Department of Computer Science and Engineering, S.D.M. College of Engineering and Technology, Dharwad**, in partial fulfilment of the degree of **Bachelor of Engineering in Computer Science and Engineering** from **Visvesvaraya Technological University, Belgaum, Karnataka**, during the academic year 2025–2026.

I also declare that I have not submitted this dissertation to any other university for the award of any other degree.

Place: Dharwad

Date: 16th December 2025

Name of Student Signature with Date

1) Abhilash Pyati (2SD23CS002)

2) Karan Kumbar (2SD23CS041)

3) Naveen Gurikar (2SD23CS061)

4) Samarth Sanikop (2SD23CS086)

ACKNOWLEDGEMENT

We express our sincere gratitude to **Visvesvaraya Technological University (VTU), Belagavi**, for providing us the opportunity to carry out this minor project as part of the Bachelor of Engineering curriculum in Computer Science and Engineering.

We would like to convey our heartfelt thanks to our respected **Project Guide, Dr. Sandhya S V**, for her constant encouragement, invaluable guidance, technical insights, and constructive suggestions throughout the course of this project. Her support and motivation played a crucial role in the successful completion of this work.

We are also thankful to the **Head of the Department, Department of Computer Science and Engineering**, **S.D.M. College of Engineering and Technology, Dharwad** and all the faculty members for their continuous support, timely guidance, and for providing the necessary infrastructure and learning environment required for this project.

We extend our sincere appreciation to the laboratory staff for their cooperation and assistance during the implementation and testing phases of the project.

Finally, we would like to thank our parents, friends, and well-wishers for their constant encouragement and moral support, which helped us to stay motivated and focused throughout the project duration.

ABSTRACT

Hand gesture-based interfaces have emerged as an intuitive and efficient means of human–computer interaction, enabling users to control digital systems without physical contact. This project, titled **"Design and Implementation of Hand Gesture Based Navigation System"**, focuses on developing a real-time, vision-based navigation system that interprets hand gestures to perform common computer operations such as cursor movement, clicking, scrolling, window switching, and system control functions like volume and brightness adjustment.

The proposed system uses a standard webcam to capture live video input, which is processed using computer vision and machine learning techniques. Hand detection and landmark tracking are performed using the **MediaPipe framework**, while **OpenCV** is utilized for image processing and visualization. Recognized gestures are mapped to system-level commands using Python-based automation libraries, enabling seamless interaction with the operating system.

The system supports multiple gestures, including single-hand and dual-hand gestures, and incorporates a mode-switching mechanism to differentiate between navigation controls and system settings. The implementation emphasizes real-time performance, accuracy, and user-friendly interaction without the need for specialized hardware.

This project demonstrates the feasibility of replacing traditional input devices with gesture-based controls, making it particularly useful in applications such as touchless interfaces, assistive technologies, smart environments, and human–machine interaction systems. The results indicate reliable gesture recognition and smooth navigation, highlighting the potential of hand gesture-based systems in modern computing environments.

PROBLEM STATEMENT

Traditional computer interfaces (keyboard, mouse, touchscreens) are not always suitable for all users or environments. People with limited mobility or in sterile environments need alternatives. We aim to **develop a contactless navigation/control system** that interprets natural hand gestures as input. Specifically, the system should:

- **Replace mouse/keyboard actions** with intuitive hand gestures (e.g., pinch to click, swipe to scroll).
- **Operate in real time** using a standard webcam (no specialized hardware).
- **Be robust to background and lighting** variations.
- **Allow mode switching** (e.g., switching between normal pointer mode and a volume/brightness control mode with a distinct gesture).
- **Enhance accessibility and hygiene**, benefiting users who cannot touch devices or need hands-free control.

Key problem: How to design a computer-vision-based system that accurately recognizes a diverse set of dynamic hand gestures and uses them to navigate a graphical user interface, while ensuring real-time performance and reliability.

TABLE OF CONTENTS

LIST OF FIGURES

Chapter 1 - INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Human–Computer Interaction (HCI) has evolved significantly over the years, moving from traditional input devices such as keyboards and mice toward more natural and intuitive interaction methods. Among these, **hand gesture recognition** has emerged as a promising approach that enables users to interact with computer systems in a contactless and user-friendly manner.

With advancements in computer vision and machine learning, real-time hand gesture recognition has become feasible using standard cameras and software frameworks. Gesture-based interaction is particularly valuable in scenarios where physical contact with devices is inconvenient, unhygienic, or inaccessible. Situations such as pandemic outbreaks, public environments, healthcare settings, and assistive technologies highlight the need for **touchless interaction systems**.

The motivation for this project arises from the growing demand for **contactless, intuitive, and accessible navigation systems** that can replace or supplement conventional input devices. By leveraging hand gestures, users can control computer functions naturally without requiring additional hardware. This project aims to explore and implement such a system using modern computer vision techniques, providing an effective alternative to traditional interaction methods.

1.2 PROBLEM STATEMENT

Traditional computer navigation relies heavily on physical input devices such as keyboards, mice, and touchscreens. These devices may not always be suitable in environments requiring strict hygiene, for users with physical disabilities, or in public systems shared by many individuals. Additionally, touch-based interfaces can become inefficient or unsafe in high-contact scenarios such as public kiosks and vending machines.

The problem addressed in this project is the **lack of an efficient, real-time, and reliable gesture-based navigation system** that allows users to control computer operations without physical contact. Existing systems often require specialized hardware, lack robustness, or fail to provide smooth real-time interaction.

Therefore, there is a need to design and implement a **hand gesture based navigation system** that can accurately recognize gestures using a standard webcam and translate them into navigation and system control actions in real time.

1.3 OBJECTIVES OF THE PROJECT

The primary objective of this project is to design and implement a real-time hand gesture based navigation system that enables intuitive and contactless interaction with a computer. The objectives are categorized as follows:

1.3.1 Technical Objectives

- To implement real-time hand detection and landmark extraction using computer vision techniques.
- To recognize static and dynamic hand gestures accurately using landmark-based analysis.
- To integrate gesture recognition with operating system controls using software automation libraries.
- To ensure real-time performance with minimal latency using standard hardware.

1.3.2 Functional Objectives

- To enable gesture-based cursor movement, clicking, scrolling, and drag-and-drop operations.
- To provide mode switching between navigation controls and system controls such as volume and brightness.
- To support single-hand and dual-hand gestures for different control operations.
- To provide visual feedback through a heads-up display for improved usability.

1.3.3 User Experience Objectives

- To design an intuitive and easy-to-learn gesture interaction system.
- To reduce dependency on physical input devices.
- To enhance accessibility for users with physical limitations.
- To provide a smooth, responsive, and user-friendly interaction experience.

1.4 SCOPE AND LIMITATIONS

1.4.1 Scope of the Project

- The system supports real-time hand gesture recognition using a standard webcam.
- It enables control of mouse operations, scrolling, window navigation, and basic system settings.
- The system can be extended to applications such as public kiosks, assistive technologies, smart environments, and contactless interfaces.
- The solution is software-based and does not require specialized sensors or hardware.

1.4.2 Limitations of the Project

- The accuracy of gesture recognition may be affected by poor lighting conditions or occlusion.
- The system performance depends on camera quality and system processing capability.
- Extremely complex or overlapping gestures are not supported in the current implementation.
- Continuous usage may cause user fatigue due to prolonged hand movements.
- The system is currently designed for single-user interaction at a time.

Chapter 2 - LITERATURE SURVEY

Recent years have seen a surge of research in vision-based hand gesture recognition (HGR), driven by applications in HCI, robotics, sign language interpretation, gaming, and assistive technology. This survey focuses on advances in HGR over the last 5–7 years, with an emphasis on real-time systems and interaction applications.

2.1 DEEP LEARNING DOMINANCE

Modern HGR systems overwhelmingly use deep neural networks. Convolutional Neural Networks (CNNs) are the backbone for learning spatial features from images. Surveys report that around 68% of recent studies use CNNs (often hybridized with RNNs or LSTMs) for gesture classification. Hybrid models (CNN+LSTM) leverage CNNs to extract spatial features and LSTMs to model temporal evolution of gestures. More recently, Transformers and attention-based models have emerged to capture long-range temporal context and sequential patterns. For example, Hu *et al.* (2023) demonstrated that combining a CNN with a Vision Transformer improved recognition of dynamic gestures. Reza Jalayer *et al.* (2025) also note that CNNs and RNNs dominate hand gesture models, with transformers and Graph Neural Networks appearing in cutting-edge research.

2.2 LANDMARK-BASED METHODS

An influential trend is to use hand *landmark coordinates* instead of raw images. Google's MediaPipe Hands (2019) provides 21 landmark positions (wrist + finger joints) for each detected hands. Using landmarks drastically reduces data input size while retaining gesture-specific geometry. For instance, Gil-Martín *et al.* (2025) showed that models trained on normalized hand landmarks achieved high accuracy even in cluttered backgrounds, because landmarks abstract away lighting/texture. Many real-time systems (including ours) adopt MediaPipe for fast hand detection and landmark extraction. The landmarks serve as input features to simple classifiers or rule-based logic, enabling real-time performance on standard hardware. Landmark-based techniques are especially useful for touchless UI control, where only hand pose (not appearance) matters.

2.3 DATA AND DATASETS

The success of deep learning HGR relies on large, diverse datasets. Public datasets like the *EgoGesture*, *IPN Hand*, and *NVGesture* provide thousands of videos of people performing common gestures. Benítez-García *et al.* (2021) created the IPN Hand dataset with 13 gesture categories (pointing, clicking, zooming, etc.) from 50 people and ≈800,000 frames. They emphasize that such varied datasets (multiple subjects, backgrounds, devices) are crucial for robust HGR models. In practice, many systems augment data by combining synthetic

backgrounds and performing hand cropping. Our project did not use external data for training, but references such datasets as benchmarks for accuracy expectations.

2.3.1 Gesture Recognition

Data Acquisition

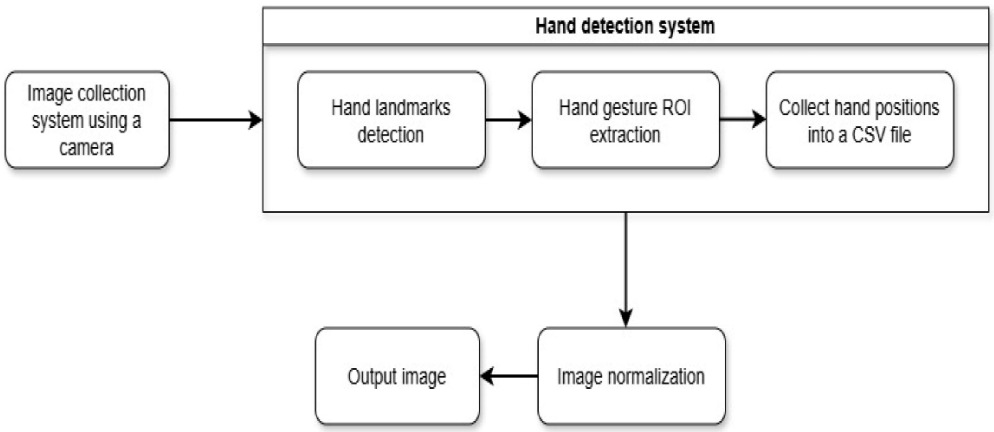


Figure 2.1: Representation of the process for collecting 6 hand gesture image sets.

Data acquisition is a fundamental stage of the proposed **Hand Gesture Based Navigation System**, as it directly influences the accuracy and responsiveness of gesture recognition. The overall system architecture for gesture acquisition and processing is composed of four major functional blocks: a camera module for capturing live hand gesture images, a hand tracking and landmark detection module, an image preprocessing module, and an output control module.

A standard webcam is used to continuously capture real-time video frames containing the user's hand gestures. These frames serve as the primary input to the system. The captured video stream is processed frame-by-frame to ensure real-time interaction. Unlike offline or dataset-based approaches, the proposed system operates entirely on live input, eliminating the need for pre-recorded gesture datasets.

The hand tracking module plays a crucial role in identifying and isolating the hand region from each video frame. This module detects the presence of one or two hands and extracts relevant spatial information such as finger positions, hand orientation, and relative distances between landmarks. The extracted hand data are then forwarded to the gesture recognition logic, where specific gestures are identified based on predefined geometric and positional rules.

To improve detection reliability, image preprocessing techniques such as background suppression, normalization, and resolution scaling are applied. Only the region of interest (ROI) containing the hand is considered for further processing, thereby reducing computational overhead and minimizing background noise. The processed data are then used to recognize gestures corresponding to navigation actions such as cursor movement, clicking, scrolling, dragging, mode switching, and system control operations like volume and brightness adjustment.

The system supports multiple dynamic and static gestures, including index-finger pointing, pinch gestures, two-finger gestures, open-palm gestures, and dual-hand gestures. These gestures are interpreted in real time and mapped to appropriate operating system navigation commands.

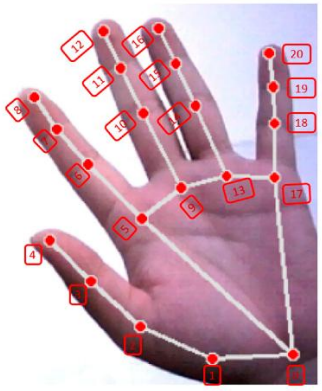
2.3.2 Hand Landmarks Detection

Hand landmark detection is performed using the **MediaPipe Hands framework**, which provides a robust and efficient solution for real-time hand tracking. MediaPipe detects the hand by first identifying the palm region in the input frame and designating it as the region of interest (ROI). Once the palm is localized, the framework extracts detailed hand landmarks within the ROI.

For each detected hand, MediaPipe identifies **21 distinct landmark points**, representing key joints and fingertips of the hand, including the wrist, finger bases, intermediate joints, and fingertip positions. These landmarks are normalized with respect to the image dimensions, allowing consistent gesture recognition regardless of hand size or distance from the camera. The system also supports simultaneous detection of up to two hands, enabling dual-hand gesture recognition.

The extracted landmark coordinates are used to compute spatial relationships such as distances between fingertips, finger extension states, and angular orientations. These geometric features form the basis for gesture classification in the proposed system. By continuously tracking landmark movement across successive frames, the system can recognize both static gestures (such as a "V" sign) and dynamic gestures (such as swipes and pinches).

The use of MediaPipe ensures high accuracy, low latency, and robustness under varying lighting conditions, making it well-suited for real-time hand gesture-based navigation applications.



0.	Wrist	11.	Middle finger DIP
1.	Thumb CMC	12.	Middle finger TIP
2.	Thumb MCP	13.	Ring finger MCP
3.	Thumb IP	14.	Ring finger PIP
4.	Thumb TIP	15.	Ring finger DIP
5.	Index finger MCP	16.	Ring finger TIP
6.	Index finger PIP	17.	Pinky MCP
7.	Index finger DIP	18.	Pinky PIP
8.	Index finger TIP	19.	Pinky DIP
9.	Middle finger MCP	20.	Pinky TIP
10.	Middle finger PIP		

Figure 2.2: Description of extracting ROI with landmarks using MediaPipe.

2.4 APPLICATIONS (NAVIGATION & ACCESSIBILITY)

A major application area is assistive devices and smart environments. For example, Nguyen *et al.* (2025) developed a **smart wheelchair** controlled by six hand gestures ("Forward", "Backward", "Left", "Right", "Stop", etc.) detected via a webcam and a YOLOv8n model on an embedded processor. They achieved **99.3% gesture recognition accuracy** in testing, enabling disabled users to navigate easily. Similarly, research into AR/VR interfaces uses HGR to navigate virtual environments (e.g. swiping through slides with a swipe gesture). These demonstrate that vision-based HGR can replace manual controls in navigation tasks. Our work targets general PC navigation (mouse, windows management) rather than mobility devices, but the underlying concept is analogous: gestures become commands (e.g. "two-finger pinch" → left-click, "palm push" → next slide).

2.5 CHALLENGES AND TRENDS

Recent reviews identify remaining gaps. Jalayer *et al.* (2025) point out that most datasets and applications focus on a single user's hands; *multi-hand* and *multi-user* scenarios are underexplored. Also, research often uses RGB cameras; depth sensors and multi-modal inputs (like combining vision with IMU) are less common. Linard Akis *et al.* (2024) note that achieving robustness in varied lighting and generalization across subjects are ongoing challenges. There is also a push toward lightweight models for real-time use on devices. In summary, recent literature shows the field is rapidly progressing with powerful CNN-based methods and large datasets. Our system builds on this by using off-the-shelf vision models (MediaPipe and basic classifiers) to implement a practical gesture navigation interface.

Key takeaway: Deep learning (especially CNN/LSTM hybrids) and landmark-based approaches dominate modern HGR research. Applications increasingly include interactive navigation (smart wheelchairs, VR, desktops). Our work leverages these insights: we use MediaPipe landmarks for fast gesture detection, apply rule-based logic akin to a trained classifier, and map gestures to navigation actions.

Chapter 3 - DESIGN

3.1 SYSTEM OVERVIEW

Figure 3 illustrates the high-level architecture of the proposed Hand Gesture Based Navigation System. The system captures live video input from a webcam, which is processed using the MediaPipe framework for real-time hand detection and landmark extraction. For each detected hand, 21 landmark points are obtained and forwarded to the gesture recognition module.

The gesture recognition module analyzes the spatial and temporal relationships between hand landmarks to identify predefined gestures and mode-switch conditions. Based on the recognized gesture, appropriate control commands such as mouse movement, clicking, scrolling, or system control actions are generated and executed using system-level libraries such as PyAutoGUI.

A dedicated mode-switch gesture allows the system to toggle between normal navigation mode and volume/brightness control mode, ensuring context-aware interaction. The system also provides real-time visual feedback by displaying detected landmarks, gesture labels, and current operating mode on the video stream, thereby improving user understanding and interaction.

Overall, the system follows a sequential processing pipeline consisting of video capture, landmark tracking, gesture analysis, and action execution, ensuring modularity and real-time performance.

3.2 LAYERED SYSTEM ARCHITECTURE

The architecture of the proposed **Hand Gesture Based Navigation System** is designed using a **layered approach** to ensure modularity, clarity, and ease of understanding. Each layer is responsible for a specific function, and the interaction between layers enables smooth real-time gesture-based navigation. This layered architecture simplifies system development, testing, and future enhancements.

3.2.1 Input Acquisition Layer

The **Input Acquisition Layer** is responsible for capturing real-time visual data from the user. A standard webcam continuously captures live video frames containing hand movements. This layer serves as the primary interface between the user and the system.

The captured frames are forwarded sequentially for further processing, ensuring uninterrupted real-time interaction. This layer abstracts the hardware dependency, allowing the system to operate with any compatible camera device.

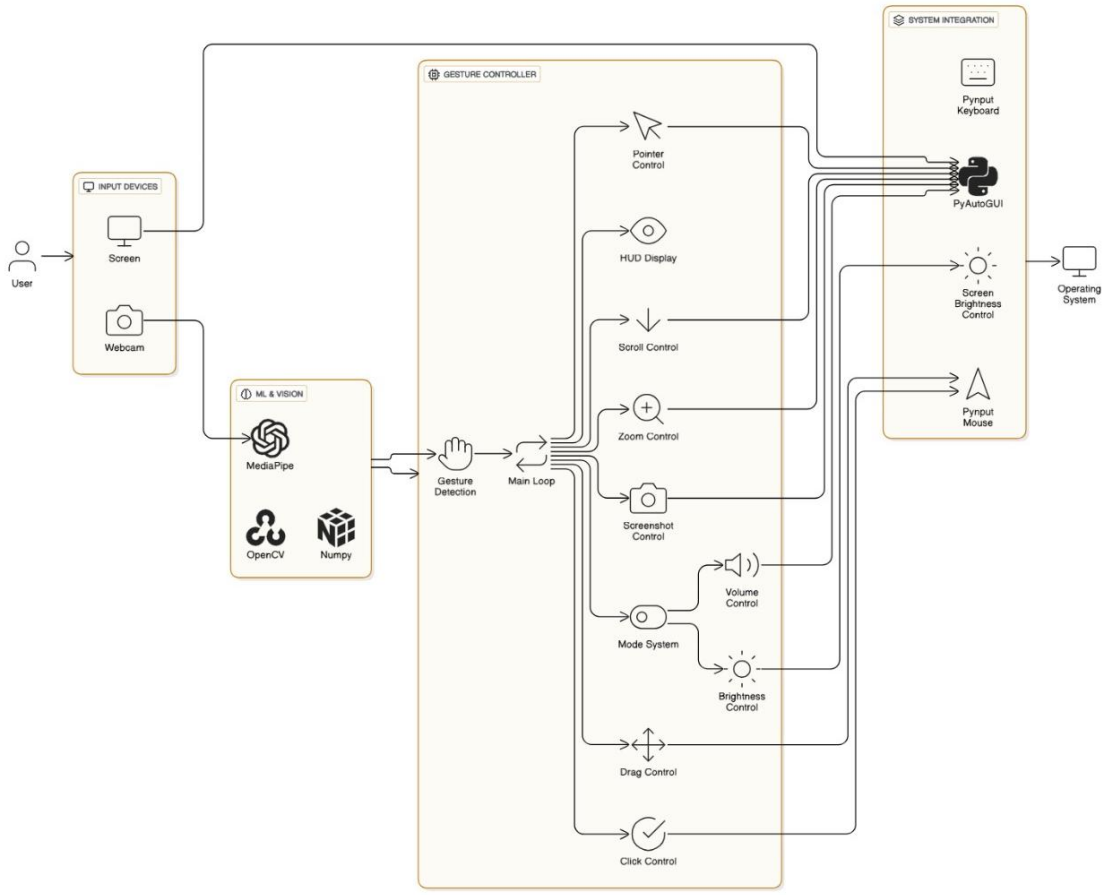


Figure 3.1: Overall system architecture of the hand gesture-based navigation system.

3.2.2 Preprocessing Layer

The **Preprocessing Layer** enhances the quality and consistency of the captured frames before gesture analysis. This layer performs operations such as frame resizing, color space conversion, and basic noise reduction to prepare the input for efficient hand detection.

By standardizing the input frames, this layer improves detection accuracy and reduces unnecessary computational overhead, enabling real-time performance.

3.2.3 Hand Detection and Landmark Extraction Layer

The **Hand Detection and Landmark Extraction Layer** is responsible for identifying the presence of hands in each video frame and extracting key hand features. This layer utilizes the **MediaPipe Hands framework** to detect hands and extract **21 landmark points per hand**, representing joints and fingertip positions.

The extracted landmarks provide precise spatial information about finger orientation, hand posture, and relative movement, forming the foundation for gesture recognition. This layer supports detection of single-hand as well as dual-hand interactions.

3.2.4 Gesture Recognition and Inference Layer

The **Gesture Recognition and Inference Layer** interprets the extracted hand landmarks to identify meaningful gestures. This layer analyses geometric relationships such as distances between fingertips, finger extension states, and relative hand movements across consecutive frames.

Temporal consistency checks and predefined thresholds are applied to ensure stable gesture recognition and to minimize false detections. Recognized gestures include pointing, pinching, scrolling, drag-and-drop gestures, and multi-finger configurations.

This layer acts as the **decision-making core** of the system, converting raw landmark data into high-level gesture commands.

3.2.5 Mode Control and Decision Layer

The **Mode Control and Decision Layer** manages system behavior based on the recognized gestures. A dedicated gesture is used to toggle between **navigation mode** and **system control mode**, ensuring context-aware operation.

In navigation mode, gestures control cursor movement, clicks, scrolling, and window navigation. In system control mode, gestures are restricted to functions such as volume and brightness adjustment. This layered decision mechanism prevents accidental actions and improves usability.

3.2.6 Action Execution Layer

The **Action Execution Layer** maps recognized gestures to corresponding operating system actions. This layer uses software automation libraries to perform tasks such as mouse movement, button clicks, scrolling, drag operations, and system-level control functions.

By abstracting OS-level interactions into a separate layer, the system ensures portability and easier adaptation to different operating environments.

3.2.7 Feedback and User Interface Layer

The **Feedback and User Interface Layer** provides real-time visual feedback to the user through a Heads-Up Display (HUD). This layer displays detected gestures, current system mode, and status information directly on the video stream.

The feedback mechanism enhances user awareness, improves learning of gesture controls, and contributes to a smoother user experience.

3.2.8 Continuous Control Loop Layer

The **Continuous Control Loop Layer** ensures that all layers operate in a synchronized and real-time manner. The system continuously processes video frames in a loop until termination, allowing uninterrupted gesture-based interaction.

This layer integrates feedback from each processing stage, enabling dynamic adjustment and refinement of system behavior during execution.

3.3 SOFTWARE PROCESS MODEL

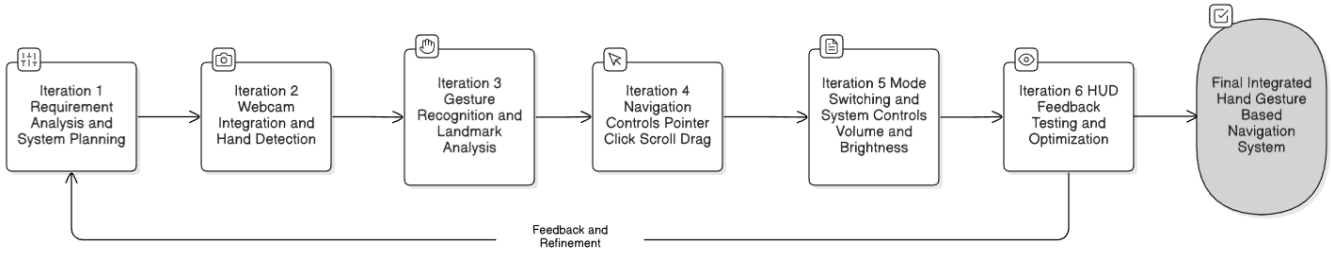


Figure 3.2: Iterative Incremental Software Development Model for the Hand Gesture Based Navigation System

The development of the **Hand Gesture Based Navigation System** follows the **Iterative Incremental Software Development Model**, which is well suited for real-time, computer vision–based applications that require continuous testing and refinement.

In this model, the system is developed through a series of iterations, where each iteration delivers a functional increment of the overall system. Instead of implementing all features at once, the project functionality was built progressively by adding and validating individual modules such as hand detection, gesture recognition, navigation controls, and system-level controls.

The initial iteration focused on **requirement analysis and system planning**, followed by the integration of the webcam and basic hand detection using the MediaPipe framework. Subsequent iterations incrementally introduced gesture recognition logic based on hand landmarks, pointer control, click and drag operations, scrolling mechanisms, and multi-hand interactions. Further iterations incorporated advanced features such as mode switching, volume and brightness control, screenshot capture, and real-time HUD feedback.

After each iteration, the system was tested using live video input to evaluate gesture accuracy, responsiveness, and user interaction behavior. Based on the observed performance, necessary refinements were made by adjusting gesture thresholds, improving detection stability, and

optimizing execution logic. This iterative feedback-driven approach ensured improved reliability and usability of the system with each development cycle.

Additionally, a **prototyping approach** was adopted during the early stages of development to experiment with different gesture mappings and interaction techniques. These prototypes helped in validating design decisions before final integration into the main system.

The Iterative Incremental Model enabled flexibility in development, early detection of issues, and continuous enhancement of system features, making it an appropriate and effective software process model for the proposed hand gesture based navigation system.

3.4 UML DIAGRAMS

UML diagrams provide a structural and behavioural representation of the system.

3.4.1 Data Flow Diagram

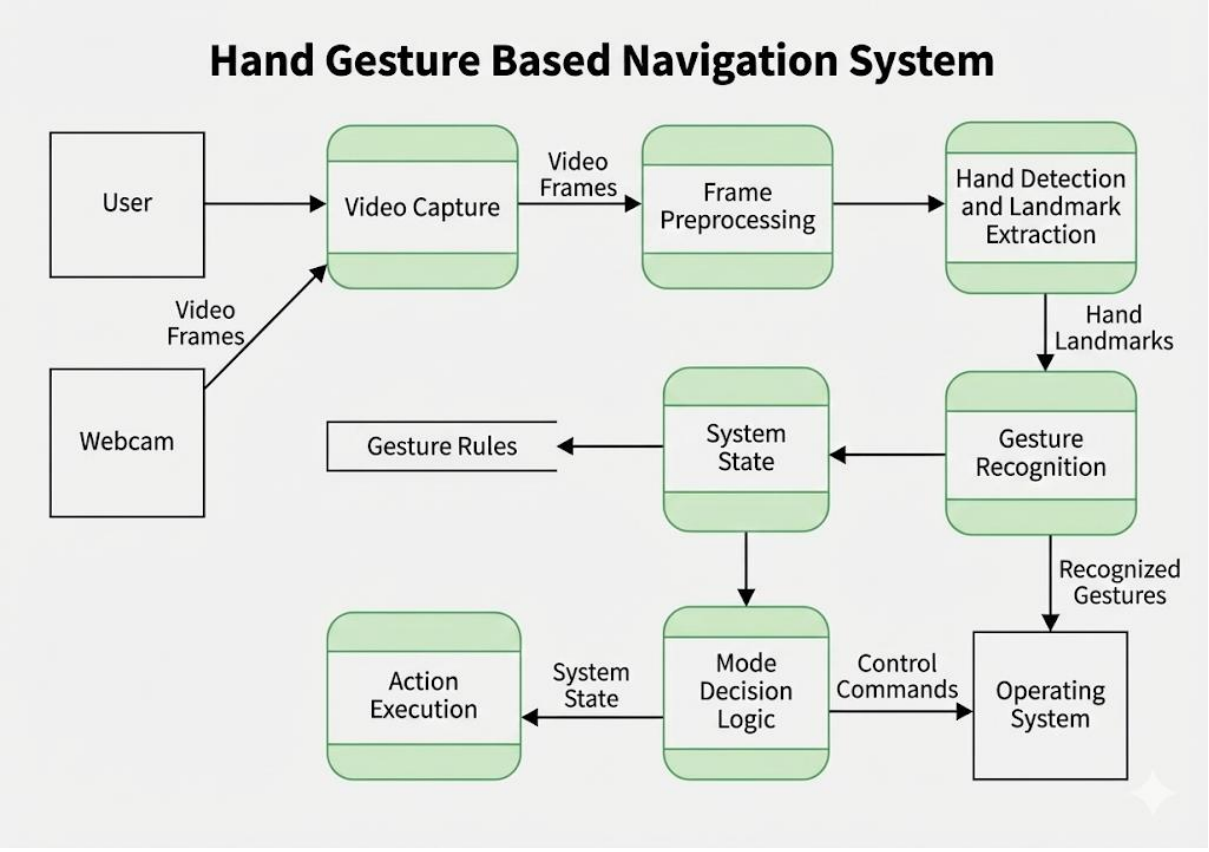


Figure 3.3: Data flow diagram of the hand gesture based navigation system

External Entities

The system interacts with three primary external entities:

1. **User –** The user performs hand gestures in front of the camera to control system navigation and actions.
2. **Webcam –** The webcam acts as the input device, continuously capturing live video frames of the user's hand movements.
3. **Operating System –** The operating system receives control commands generated by the system and executes corresponding actions such as mouse movement, clicks, scrolling, or system-level controls.

Core Processes

1. **Video Capture**: The process begins with the Video Capture module, which receives continuous video frames from the webcam. These frames represent raw visual input and form the primary data stream entering the system.
2. **Frame Preprocessing:** The captured video frames are passed to the Frame Preprocessing process. This module enhances the input data by performing operations such as resizing, noise reduction, color conversion, and normalization. Preprocessing ensures that the frames are suitable for accurate hand detection and analysis.
3. **Hand Detection and Landmark Extraction**: Pre-processed frames are then processed by the Hand Detection and Landmark Extraction module. Using computer vision techniques (e.g., MediaPipe), the system identifies the presence of a hand and extracts key hand landmarks such as finger tips, joints, and palm points. These landmarks provide a structured numerical representation of the hand's position and posture.
4. **Gesture Recognition**: The extracted hand landmarks are forwarded to the Gesture Recognition process. This module analyzes spatial and temporal patterns in the landmarks to identify predefined gestures. Recognized patterns are classified into meaningful recognized gestures (e.g., swipe, pinch, fist, or mode-switch gesture).
5. **Mode Decision Logic:** Recognized gestures are evaluated by the Mode Decision Logic process. This module determines the current operational mode of the system (for example, navigation mode or volume/brightness control mode) based on gesture input and the current system state. It generates appropriate control commands that correspond to the identified gesture and active mode.
6. **Action Execution**: The Action Execution process receives the finalized commands and executes them through system-level libraries and APIs. This results in real-time interaction with the operating system, such as cursor movement, clicks, scrolling, or other navigation actions.

Data Stores

1. **Gesture Rules**: The Gesture Rules data store contains predefined mappings between hand landmark patterns and corresponding gestures. It ensures consistency and accuracy during gesture recognition.
2. **System State**: The System State data store maintains information about the current mode and operational context of the system. It allows the system to respond dynamically to gestures based on previously executed actions or mode selections.

3.4.2 Use Case Diagram

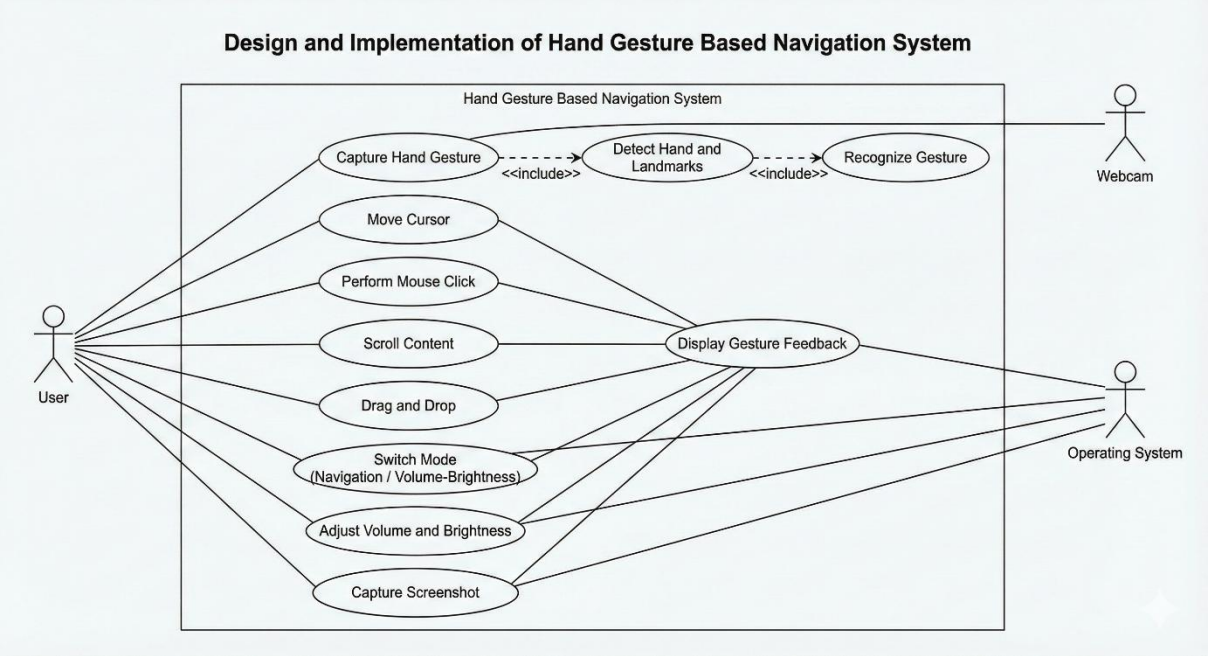


Figure 3.4: Use case diagram of the hand gesture based navigation system.

Actors: The system involves three primary actors:

1. **User:** The User is the main actor who interacts with the system by performing various hand gestures. These gestures are interpreted to control cursor movement, mouse operations, scrolling, system mode switching, and multimedia controls.
2. **Webcam:** The Webcam serves as an external input device that continuously captures real-time hand gesture data. It supports gesture acquisition and acts as a sensing interface between the physical world and the system.
3. **Operating System:** The Operating System is the execution environment that receives control commands generated by the system. It performs actions such as cursor movement, mouse clicks, scrolling, brightness adjustment, volume control, and screenshot capture.

Use Cases

The diagram includes the following use cases, each representing a distinct system function:

- **Capture Hand Gesture:** This use case initiates the system workflow by acquiring live video input of the user's hand through the webcam.
- **Detect Hand and Landmarks:** This use case extracts key hand landmarks from the captured frames using computer vision techniques. It is an essential intermediate step required for gesture recognition.
- **Recognize Gesture:** Based on extracted landmarks, this use case identifies specific gestures corresponding to predefined patterns.
- **Move Cursor:** Enables real-time cursor movement based on hand position and motion.
- **Perform Mouse Click:** Allows execution of left or right mouse clicks using specific hand gestures.
- **Scroll Content:** Enables vertical or horizontal scrolling of on-screen content using gesture-based input.
- **Drag and Drop:** Allows objects to be selected and repositioned using continuous hand movement gestures.
- **Switch Mode (Navigation / Volume-Brightness):** This use case toggles the system between normal navigation mode and system control mode, enabling contextual interpretation of gestures.
- **Adjust Volume and Brightness:** Allows the user to control system volume and screen brightness when the appropriate mode is active.
- **Capture Screenshot:** Enables screen capture functionality using a predefined gesture.
- **Display Gesture Feedback:** Provides visual feedback to the user by displaying detected landmarks, recognized gestures, or system status on the screen.

Relationships and Associations

- The User is directly associated with all interaction-based use cases, emphasizing that gestures originate from user input.
- The Webcam is associated with gesture acquisition and recognition-related use cases, highlighting its role as an input device.
- The Operating System is associated with execution-based use cases such as cursor movement, clicking, scrolling, system control, and screenshot capture.
- The use cases Detect Hand and Landmarks and Recognize Gesture are shown as included processes within gesture capture, reflecting their dependency in the gesture processing pipeline.

3.4.3 Activity and Sequence Diagram

- **Activity Diagram**

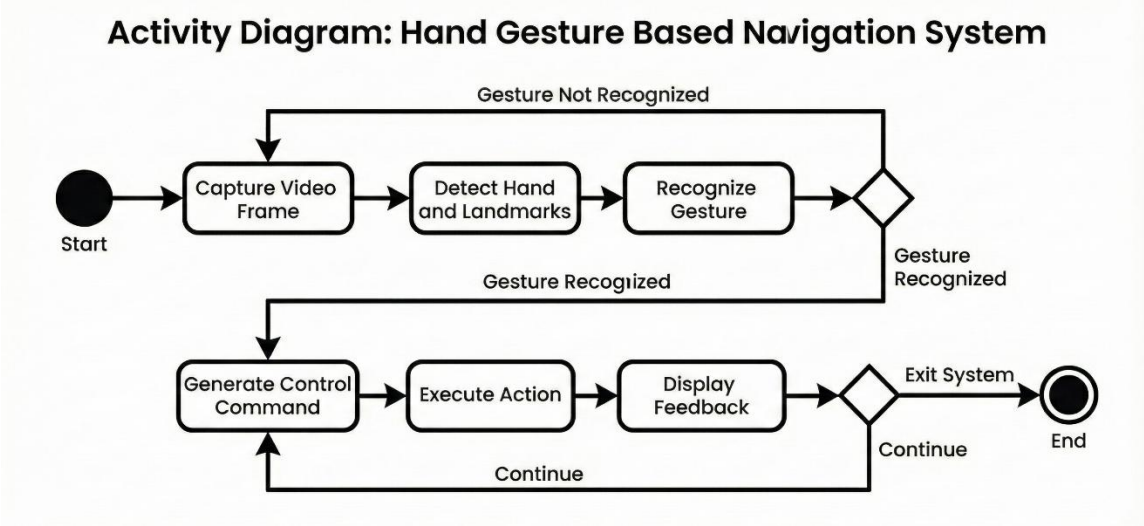


Figure 3.5: Activity diagram of the hand gesture based navigation system.

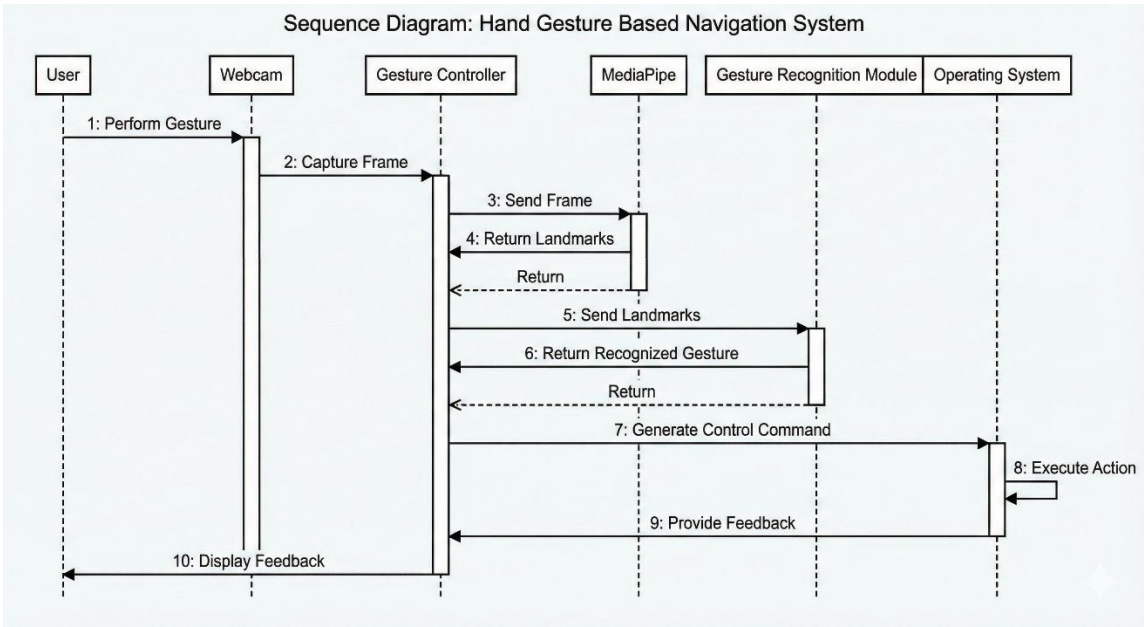


Figure 3.6: Sequence diagram of the hand gesture based navigation system.

- The sequence diagram illustrates the interaction over time among system components during gesture-based navigation.
- The process begins when the User performs a hand gesture in front of the webcam.
- The Webcam captures the video frame containing the hand gesture.
- The captured frame is sent to the Gesture Controller, which coordinates further processing.
- The Gesture Controller forwards the frame to MediaPipe for hand detection and landmark extraction.

- MediaPipe extracts 21 hand landmarks per detected hand and returns them to the Gesture Controller.
- The Gesture Controller sends the extracted landmarks to the Gesture Recognition Module.
- The Gesture Recognition Module analyzes landmark geometry and temporal patterns to identify the gesture.
- The recognized gesture is returned to the Gesture Controller.
- Based on the gesture and current mode, the Gesture Controller generates a control command.
- The control command is sent to the Operating System.
- The Operating System executes the requested action, such as mouse movement, click, scroll, or system control.
- Execution feedback is sent back to the Gesture Controller.
- The Gesture Controller displays visual feedback (gesture name, landmarks, mode) to the user.
- The sequence runs in a continuous loop, enabling real-time gesture-based interaction.

3.5 METHODOLOGY

The methodology of the proposed **Hand Gesture Based Navigation System** describes the systematic and step-by-step approach adopted to design, implement, and evaluate the system. The methodology is structured to ensure real-time performance, accurate gesture recognition, and seamless interaction with the operating system using hand gestures.

The process begins with **real-time video acquisition**, where a standard webcam continuously captures live video frames of the user's hand. The webcam serves as the primary input device, and the captured frames are processed sequentially to enable uninterrupted gesture-based interaction.

Each captured video frame undergoes **preprocessing** to improve detection efficiency. Preprocessing operations include frame resizing, color space conversion from BGR to RGB, and noise reduction. These steps ensure compatibility with the computer vision framework and enhance the accuracy of subsequent hand detection.

The pre-processed frames are then passed to the **hand detection and landmark extraction module**, implemented using the MediaPipe Hands framework. MediaPipe detects the presence of one or two hands in the frame and extracts **21 hand landmark points** per detected hand, representing key joint positions and fingertips. These landmarks provide precise spatial information about hand posture and finger orientation.

Next, the system performs **gesture recognition** by analyzing geometric relationships between the extracted landmarks. Distances between fingertips, finger extension states, and relative hand positions are computed to identify specific gestures such as pointing, pinching, scrolling,

and multi-finger gestures. Temporal consistency checks are applied to reduce false detections and ensure gesture stability across consecutive frames.

A **mode determination mechanism** is incorporated to enhance usability and prevent accidental operations. A predefined gesture is used to toggle between **full navigation mode** and **restricted system control mode**. In full navigation mode, gestures control cursor movement, click, drag, scroll, and zoom operations. In system control mode, gestures are limited to volume and brightness adjustments. This decision logic is evaluated continuously within the main execution loop.

Once a gesture is recognized and the active mode is determined, the system proceeds to the **action execution phase**. Recognized gestures are mapped to corresponding operating system actions using Python-based automation libraries. These actions include mouse movement, button clicks, scrolling, drag-and-drop operations, application switching, and system-level controls such as volume and brightness adjustment.

To enhance user awareness and interaction, a **Heads-Up Display (HUD)** is overlaid on the video stream. The HUD provides real-time feedback by displaying detected gestures, current operating mode, and system status. This visual feedback helps users understand system responses and improves overall usability.

The entire process operates within a **continuous real-time loop**, allowing the system to process successive video frames until termination. Each iteration of the loop incorporates feedback from gesture recognition performance, enabling refinement of gesture thresholds and control logic for improved accuracy and responsiveness.

This structured methodology ensures modularity, real-time performance, and reliability, making it well-suited for implementing a hand gesture based navigation system that replaces conventional input devices with intuitive, touchless interaction.

3.6 System Modes and Functions:

The system has two main modes of operation:

- **Normal Navigation Mode:** Default mode. Gestures control the pointer, clicking, drag-and-drop, window switching (Alt+Tab via left/right swipe), scrolling, and opening files (e.g. an open-palm gesture). This mode maps to typical desktop actions.
- **Volume/Brightness Control Mode:** Toggled by a "V" sign gesture. In this mode, similar gestures control system settings: e.g. swipe left/right with open palms changes volume or brightness, two-hand pinch might change screen zoom, etc. We implemented two sub-modes toggled by horizontal swipes: one for volume, one for brightness, indicated by HUD text.

Switching between modes ensures that gestures have context-sensitive effects. The "V" sign serves as a safety to prevent accidental mode changes.

3.7 Interface Diagram:

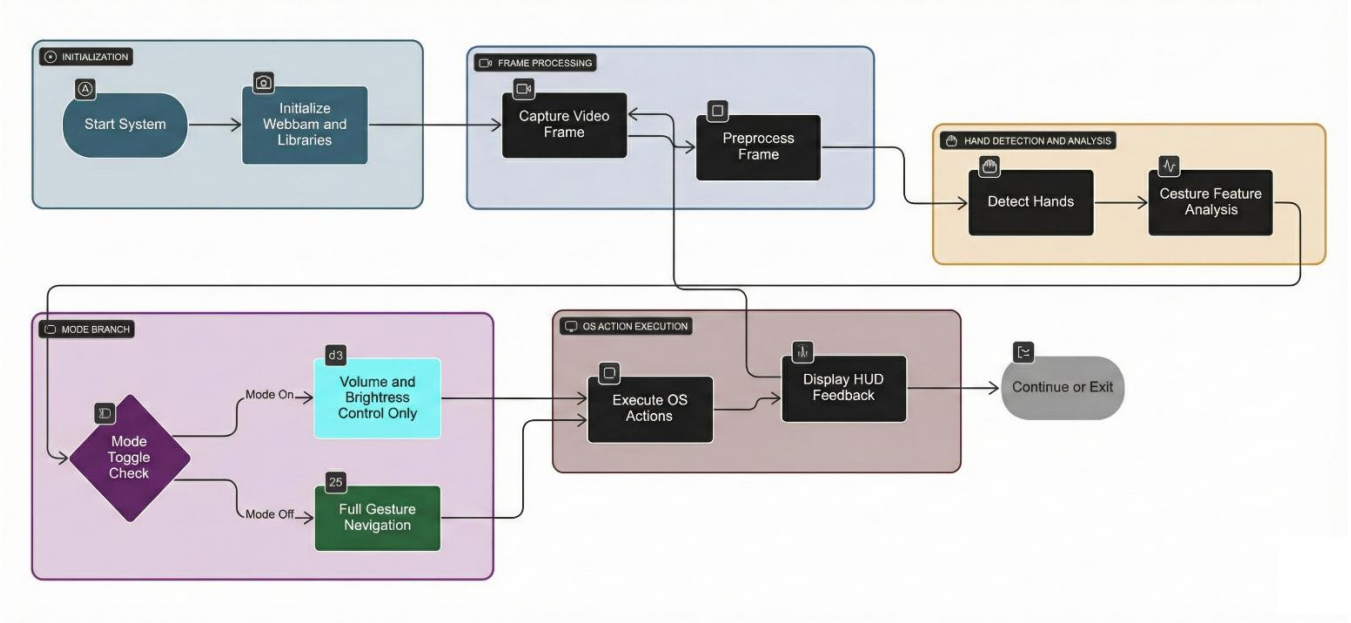


Figure 3.7: Flowchart representing the execution logic of the hand gesture based navigation system.

Chapter 4 - PROJECT SPECIFIC REQUIREMENTS

The project has the following functional and non-functional requirements:

- **Functional Requirements:**
    - The system shall recognize at least the following gestures: index-finger pinch, middle-finger pinch, index+middle extended (scroll), two-finger "V" sign, open palm, double-hand fist.
    - The system shall execute mouse movements (pointer control) when the index finger is extended.
    - The system shall execute left-click when a quick pinch of thumb and index finger is detected.
    - The system shall execute right-click when a pinch of thumb and middle finger is detected.
    - The system shall execute double-click on a fast repeated pinch gesture.
    - The system shall allow click-and-drag when the pinch is held (drag) and released (drop).
    - The system shall recognize a two-handed gesture for switching mode.
    - The system shall allow scrolling up/down when a specific gesture is held (e.g. two fingers together).
    - The system shall work in real time (min 15 FPS processing).
    - The user shall receive visual feedback (on-screen text) for each recognized gesture/action.
    - The system shall run on a standard PC with a USB camera; no special hardware needed.

- **Non-functional Requirements:**
    - **Performance:** Gesture recognition latency must be low (sub-100ms from frame to action) to feel instantaneous to the user.
    - **Reliability:** System should maintain tracking even if hands move slightly; sporadic false positives must be minimized by requiring distinct gesture poses.
    - **Portability:** Written in Python, using cross-platform libraries where possible. The implementation uses OpenCV, MediaPipe, and standard GUI libraries (pyautogui, etc.) to ensure compatibility with Windows 10 or later.
    - **Usability:** The gestures chosen are standard and should be easy to remember (e.g., pinch = click). The overlay helps learn the mapping.

- **Safety:** The system includes a short activation delay (e.g. requiring gesture to be held 0.5 seconds for toggle) to avoid accidental commands.

- **Software Environment:**
  - Python 3.8+ with modules: opencv-python, mediapipe, pyautogui, pynput, screen_brightness_control, and numpy.
  - Tested on Windows 10; should also work on Linux.

- **Hardware Environment:**
  - 1.5+ GHz CPU (modern laptop or desktop is sufficient).
  - Standard USB webcam (min 640×480 resolution, ~30fps).
  - No GPU required for the basic MediaPipe tracking (it can run on CPU).

Overall, the requirements ensure the system integrates established gesture algorithms (pinch, swipe, etc.) into a user-friendly PC navigation tool. The literature confirms these mappings; for instance, index-finger pinch as left-click is a common convention, and two-hand gestures for mode toggling are used in other interfaces. We verified during development that the implemented gestures satisfy the intended control functions.

Chapter 5 - IMPLEMENTATION

The implementation phase converts the proposed design into a functional Hand Gesture Based Navigation System. The system is implemented using Python and integrates computer vision techniques with system automation libraries to enable real-time, touchless interaction with the operating system. OpenCV and MediaPipe are used for video capture and hand landmark detection, while PyAutoGUI and pynput are used for cursor and system-level control.

The complete implementation is provided in the core source file Final_Code.py. This section explains the major modules and presents key code snippets that illustrate critical implementation logic.

5.1 Hand Tracking and Landmark Extraction Module

This module captures live video input from a webcam and detects hand landmarks in real time. Video frames are captured using OpenCV and converted to RGB format before being processed by MediaPipe Hands.

Key steps:

1. Capture frame using OpenCV
2. Convert frame from BGR to RGB
3. Process frame using MediaPipe Hands
4. Extract and draw landmarks

Code Snippet: Frame Capture and Landmark Detection

```
ret, frame = cap.read()
if not ret:
    break
image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
result = hands.process(image_rgb)

if result.multi_hand_landmarks:
    for handLms in result.multi_hand_landmarks:
        mp.solutions.drawing_utils.draw_landmarks(
            frame, handLms, mp_hands.HAND_CONNECTIONS
        )
```

MediaPipe extracts 21 landmarks per hand, representing fingertips, joints, and palm points.

These landmarks are forwarded to the gesture recognition module.

5.2 Gesture Recognition and Decision Logic Module

Gesture recognition is implemented using **rule-based geometric analysis** instead of machine learning, enabling low-latency and deterministic control.

The system evaluates:

- Normalized distances between landmarks
- Finger extension states
- Relative finger positions over time

Helper functions are used to compute distances and determine whether fingers are extended.

Example Logic: Finger Extension Check

```
def finger_extended_strict(hand, tip_id, mcp_id):
    return hand.landmark[tip_id].y < hand.landmark[mcp_id].y
```

This logic enables detection of gestures such as pointing, pinching, scrolling, and mode switching.

5.3 Cursor Control and Pointer Movement Module

When only the index finger is extended, the system enters **pointer mode**. The fingertip position is mapped to screen coordinates and used to move the mouse cursor.

Code Snippet: Pointer Movement

```
if pointer_mode:
    finger_pos =
hand.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP]
    screen_x = int(finger_pos.x * SCREEN_WIDTH)
    screen_y = int(finger_pos.y * SCREEN_HEIGHT)
    pyautogui.moveTo(screen_x, screen_y, duration=0.01)
```

A visual marker is drawn at the fingertip position to provide feedback.

5.4 Click, Drag, and Scroll Action Module

5.4.1 Click Detection Using Pinch Gesture

A mouse click is detected when the distance between the thumb and index finger falls below a predefined threshold. Timing logic is used to differentiate between clicks and drag actions.

Code Snippet: Left Click Detection

```
idx_now = norm_dist(index_tip, thumb_tip) < PINCH_TOUCH_THRESH
if idx_now and not index_pinch_prev:
```

```
        index_pinch_start = time.time()

    if not idx_now and index_pinch_prev:
        if (time.time() - index_pinch_start) < FAST_CLICK_TIME:
            mouse.click(Button.left, 1)
            set_hud("LEFT CLICK")
        index_pinch_start = 0

    index_pinch_prev = idx_now
```

5.4.2 Drag and Drop Operation

If a pinch gesture is held beyond a predefined duration, the system initiates a drag operation by simulating a mouse press. Releasing the pinch ends the drag.

This enables natural click-and-drag behavior using hand gestures.

5.4.3 Scroll Control

Scrolling is enabled when both the index and middle fingers are extended. Vertical hand movement triggers scroll actions with a controlled delay to prevent rapid scrolling.

```
if scroll_up_pose:
    pyautogui.scroll(SCROLL_STEP)
    set_hud("SCROLL UP")
```

5.5 Mode Switching and System Control Module

To avoid accidental system actions, the system uses a **mode-based control strategy**. A predefined **"V" sign gesture** toggles between:

- Normal Navigation Mode
- Volume/Brightness Control Mode

The gesture must be held for a short duration before switching modes. A HUD indicator confirms the transition.

5.6 Screenshot and Multi-Hand Gesture Module

The system supports limited two-hand gestures. When both hands form a fist simultaneously, a screenshot is captured using PyAutoGUI and saved as a PNG file.

Before executing two-hand actions, the system verifies that two hands are detected to ensure reliability.

5.7 User Feedback and HUD Module

A Heads-Up Display (HUD) is rendered on the live video stream using OpenCV. The HUD displays:

- Recognized gesture name
- Current operating mode
- Action status messages

This visual feedback improves usability and helps users understand system responses in real time.

5.8 Software and Hardware Requirements

Software Requirements:

- Operating System: Windows / Linux
- Programming Language: Python 3.x
- Libraries: OpenCV, MediaPipe, NumPy, PyAutoGUI, pynput

Hardware Requirements:

- Standard USB webcam
- Desktop or laptop computer

5.9 IMPLEMENTATION CHALLENGES

During development, we addressed issues such as:

- **Noise and Stability:** Hand landmarks can jitter; we used simple smoothing (e.g., low-pass filtering of coordinates) and thresholds (e.g., requiring gestures to be held briefly) to avoid false triggers.
- **Lighting Variations:** MediaPipe's model is robust to lighting changes; we found it worked well under indoor lighting without extra calibration.
- **Mode Ambiguity:** Ensuring the V-sign reliably toggles modes required tuning (e.g., finger gap threshold and hold time). We implemented a grace period (HUD progress bar) during the hold as visual feedback, as seen in code [15†L798-L807].
- **Coordination Between Hands:** Our system handles up to two hands; some gestures (zoom) use two hands, while others assume one. The code carefully checks if n >= 2 (two detected hands) for two-hand gestures, otherwise ignores one-hand gestures.

Despite these, the implementation successfully ties the provided algorithms (Mediapipe tracking + PyAutoGUI actions) into a working navigation tool.

Chapter 6 - RESULTS AND DISCUSSION

6.1 Overview of Experimental Results

The proposed **Hand Gesture Based Navigation System** was tested using real-time video input captured through a webcam under normal indoor lighting conditions. The system was evaluated based on its ability to accurately detect hand landmarks, recognize predefined gestures, and execute corresponding navigation and system control actions in real time. Screenshots captured during live execution are presented in this section to visually demonstrate the effectiveness of each implemented gesture.

6.2 Gesture Recognition Results

The following subsections present **screenshots of all implemented gestures**, captured during real-time system execution. Each screenshot illustrates successful gesture detection along with the corresponding system response and visual feedback.

6.2.1 Pointer (Cursor Movement) Gesture

The pointer gesture, where the index finger is extended while other fingers are folded, was used to control mouse cursor movement.
The system accurately mapped the fingertip position to screen coordinates, enabling smooth and responsive cursor navigation.

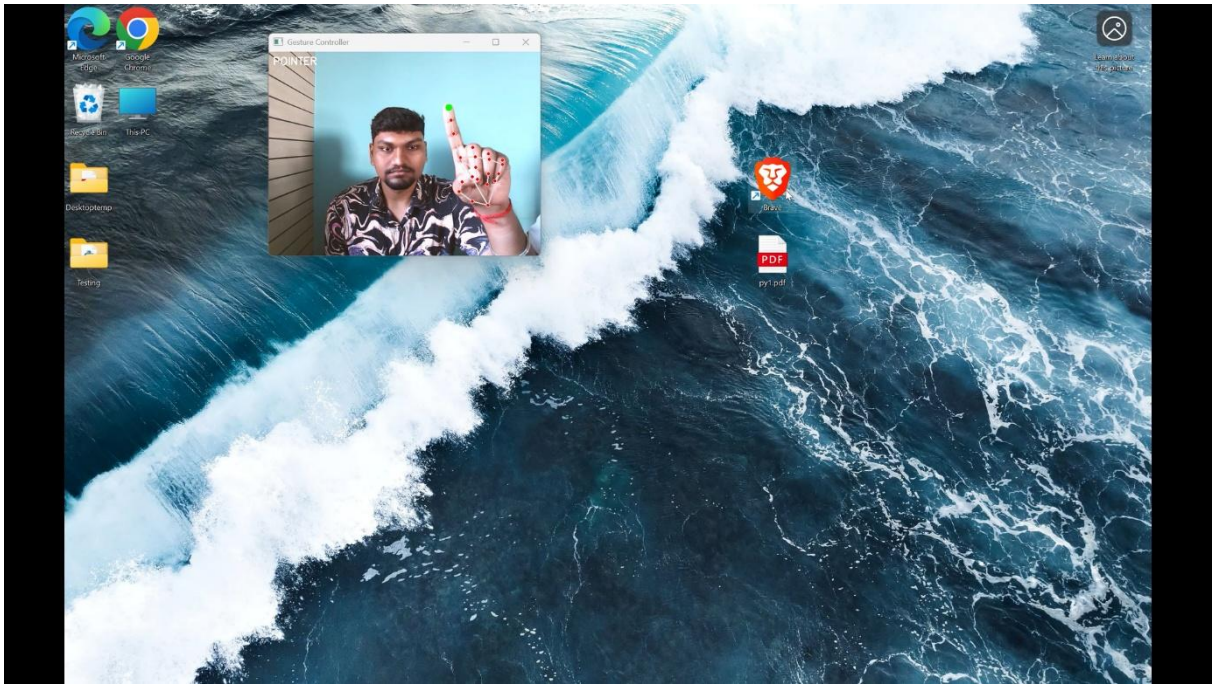


Figure 6.1: Screenshot showing pointer gesture used for cursor movement.

6.2.2 Left Click Gesture (Index-Finger Pinch)

The left-click operation was performed using an index-finger pinch gesture. The system successfully detected the pinch gesture and generated a mouse click event, as indicated by the on-screen feedback.

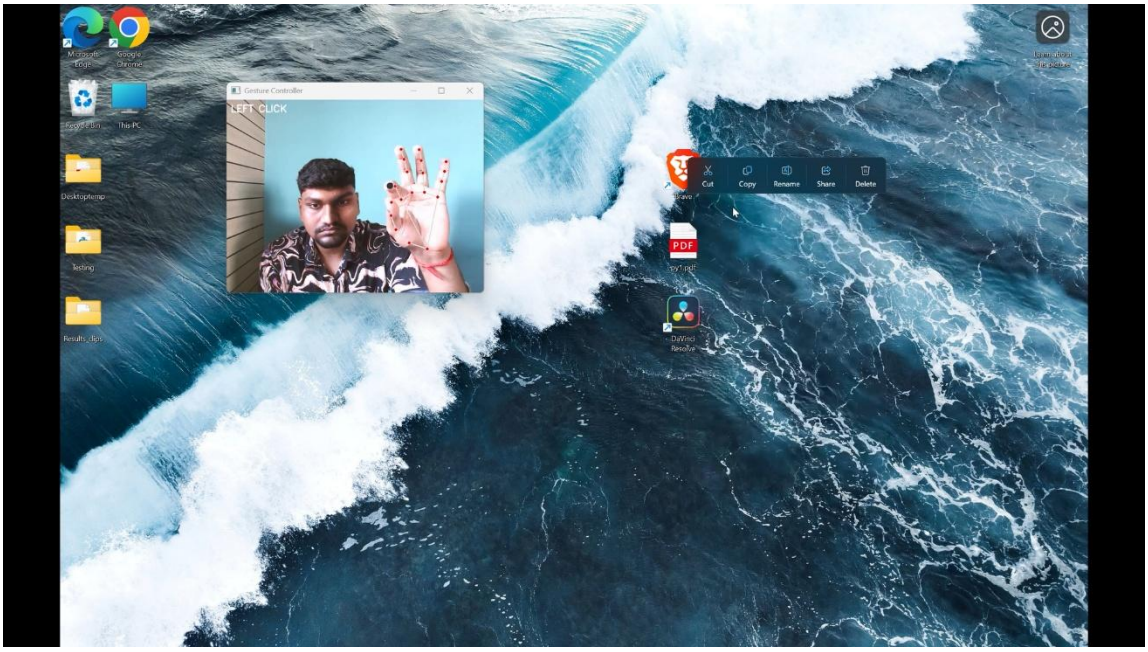


Figure 6.2: Screenshot showing index-finger pinch gesture for left-click operation.

6.2.3 Right Click Gesture (Middle-Finger Pinch)

A pinch between the thumb and middle finger was used to perform a right-click operation. The system reliably differentiated this gesture from the left-click gesture and executed the correct mouse action.

Figure 6.3: Screenshot showing middle-finger pinch gesture for right-click operation.

6.2.4 Drag and Drop Gesture

The drag-and-drop operation was achieved by holding the index-finger pinch gesture for a predefined duration.
The system maintained the click state during the drag operation and released it correctly upon gesture release.

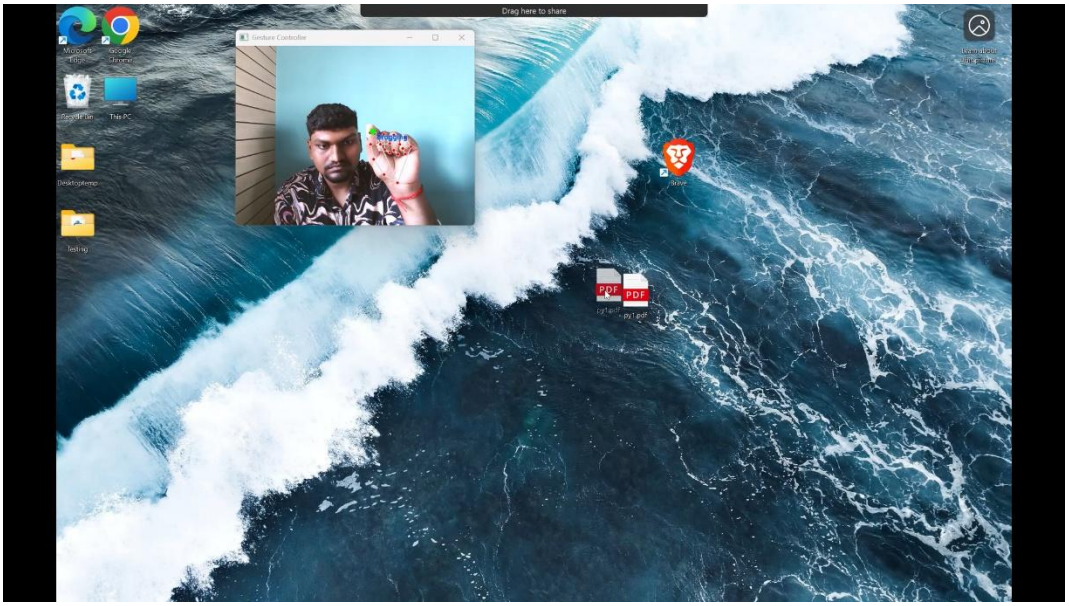


Figure 6.4: Screenshot showing drag-and-drop operation using hand gesture.

6.2.5 Scroll Gesture

Scrolling was performed using a two-finger gesture with the index and middle fingers extended together.Vertical hand movement resulted in smooth scrolling of content, demonstrating stable gesture recognition.

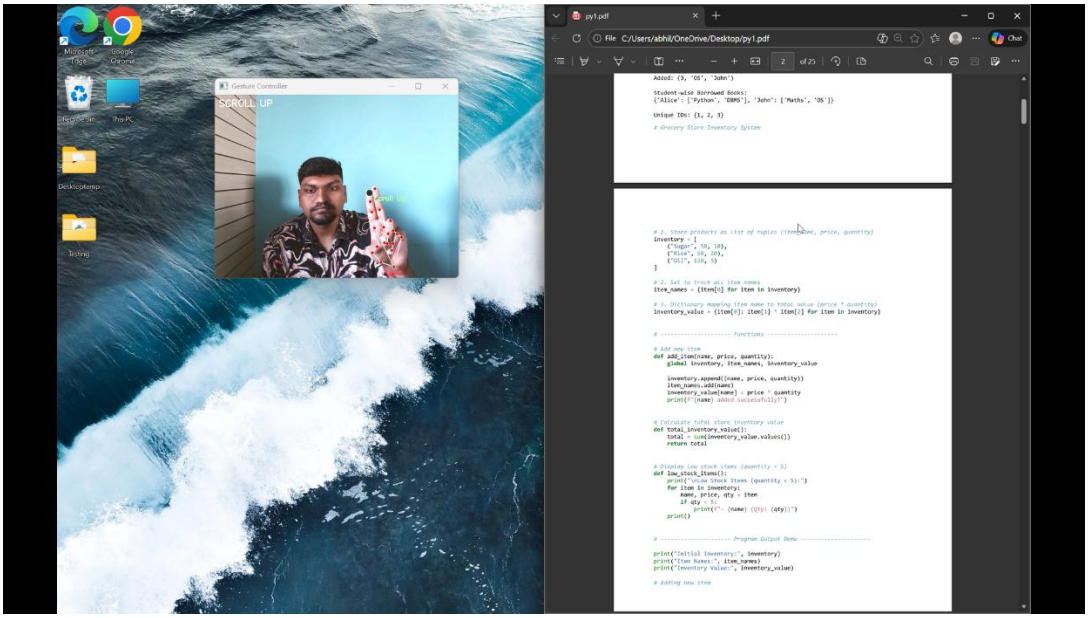


Figure 6.5: Screenshot showing scroll-up gesture for vertical content navigation.

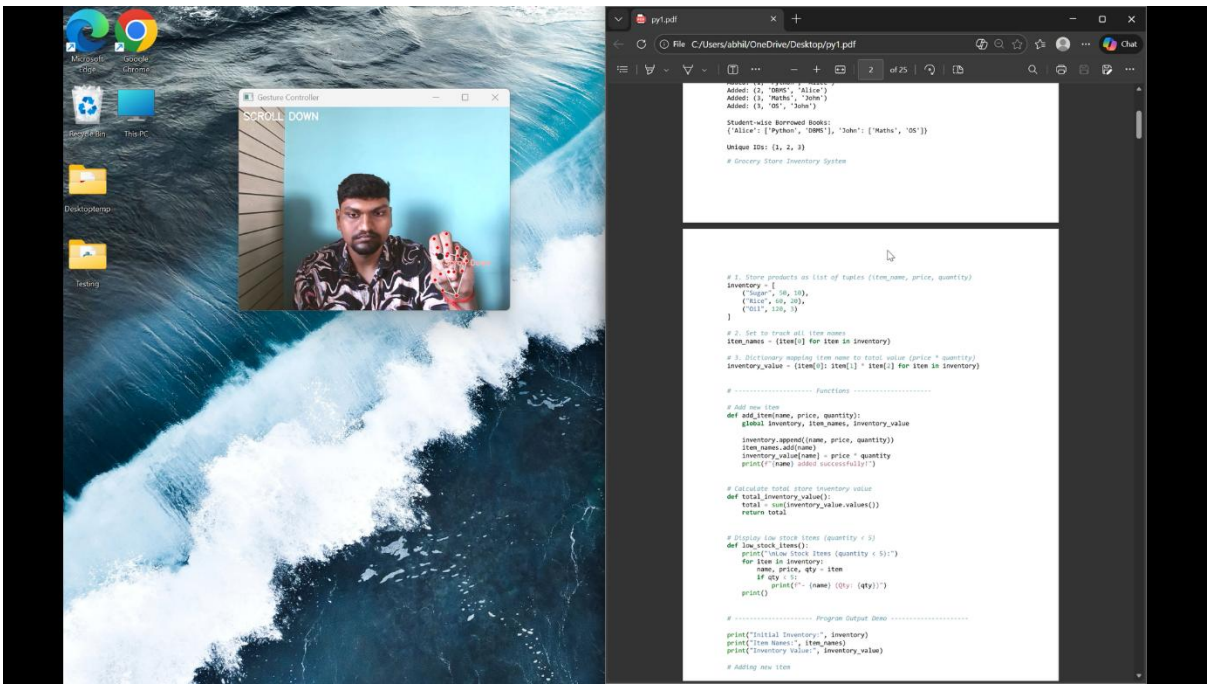


Figure 6.6: Screenshot showing scroll-down gesture for vertical content navigation.

6.2.6 Zoom Gesture

Zoom-in and zoom-out operations were implemented using a dual-hand gesture, where the distance between hands controlled the zoom level. The system responded effectively to changes in hand separation.

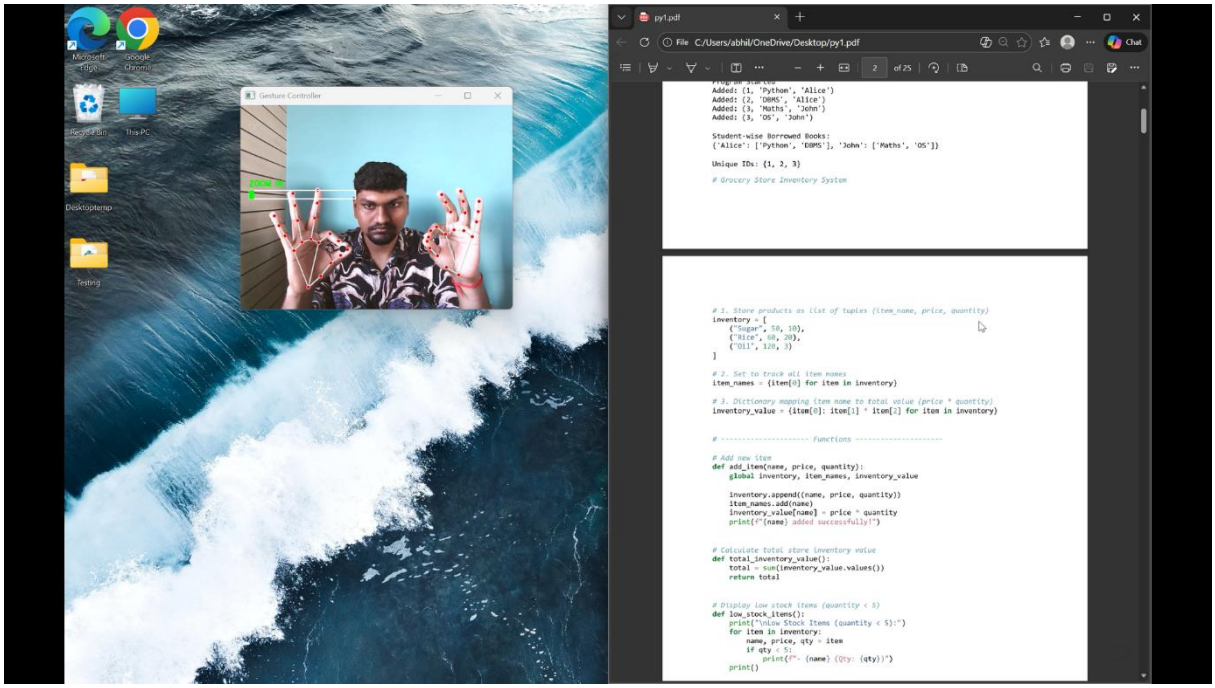


Figure 6.7: Screenshot showing zoom-in control using two-hand gesture.

6.2.7 Mode Switching Gesture

A two-finger "V" sign gesture was used to toggle between **navigation mode** and **volume/brightness control mode**.
The system correctly detected the gesture and displayed the active mode on the HUD.

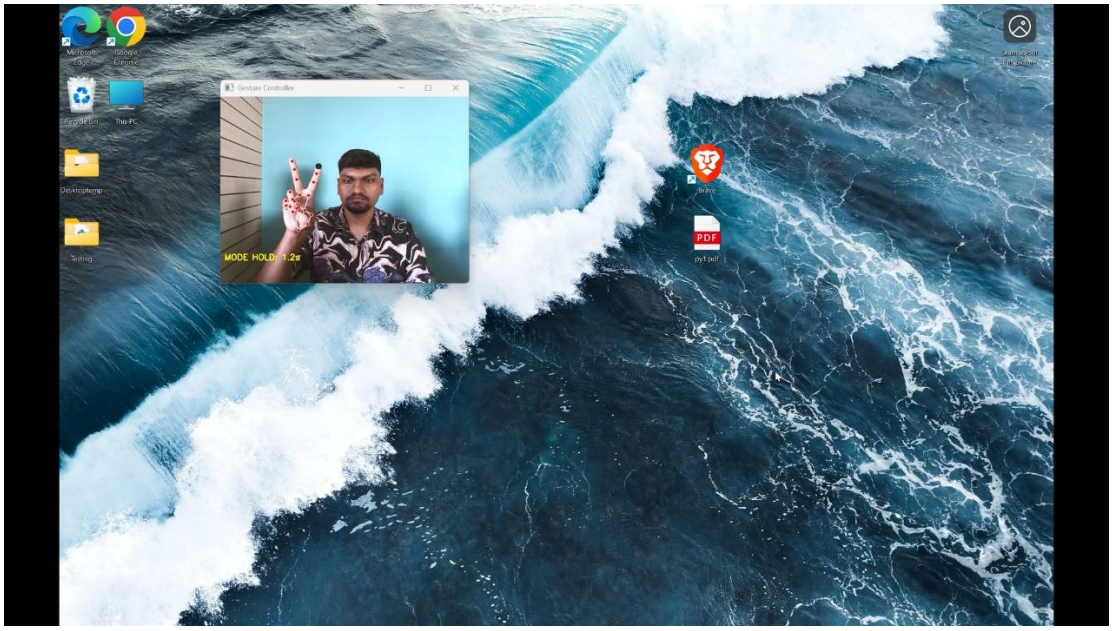


Figure 6.8: Screenshot showing mode switching using V-sign gesture.

6.2.8 Volume and Brightness Control Gesture

In system control mode, vertical hand movement was used to adjust volume and brightness levels.
The system provided immediate feedback, confirming successful control execution.

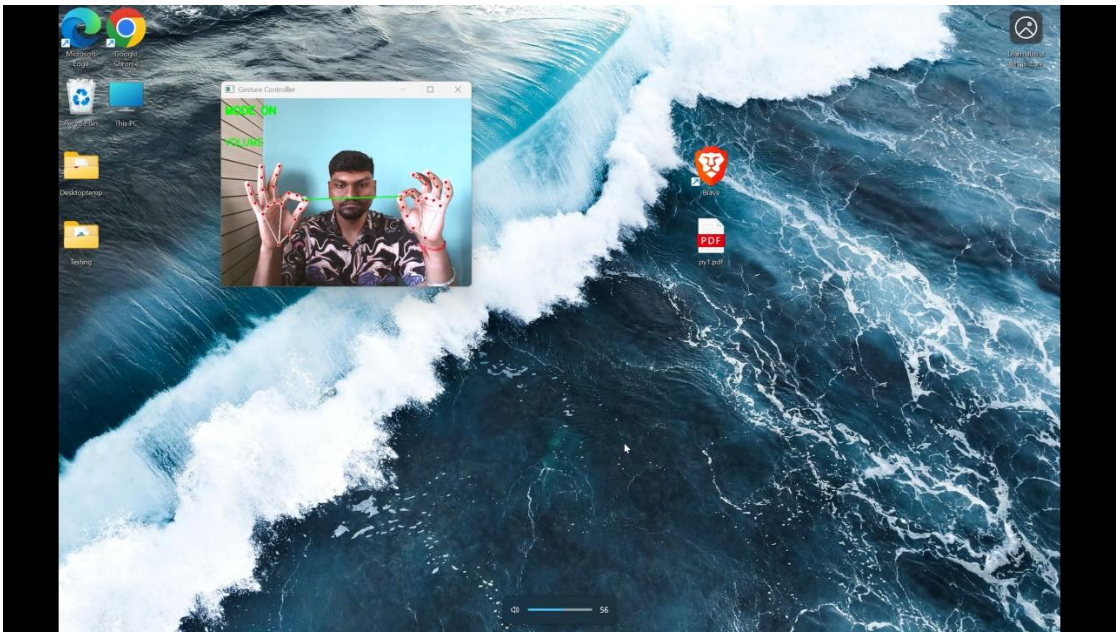


Figure 6.9: Screenshot showing gesture-based volume control.

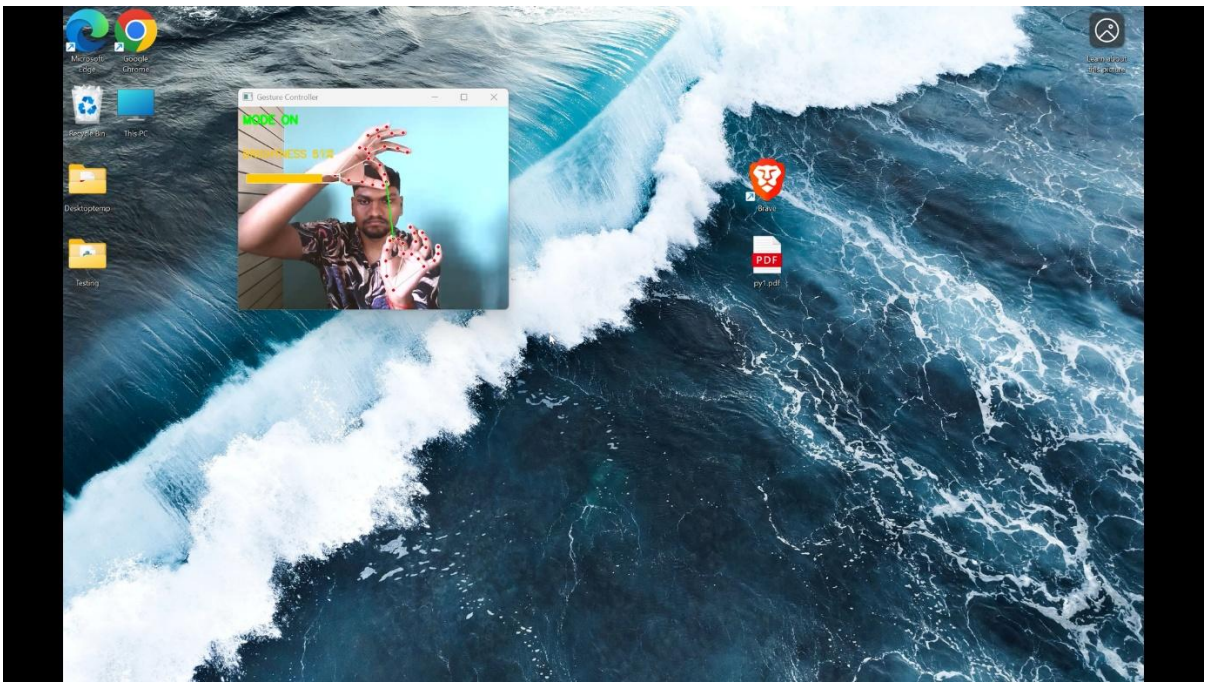


Figure 6.10: Screenshot showing gesture-based brightness control.

6.2.9 Screenshot Capture Gesture

A dual-hand closed-fist gesture was used to capture screenshots. The system successfully detected the gesture after a short hold duration and saved the screenshot.

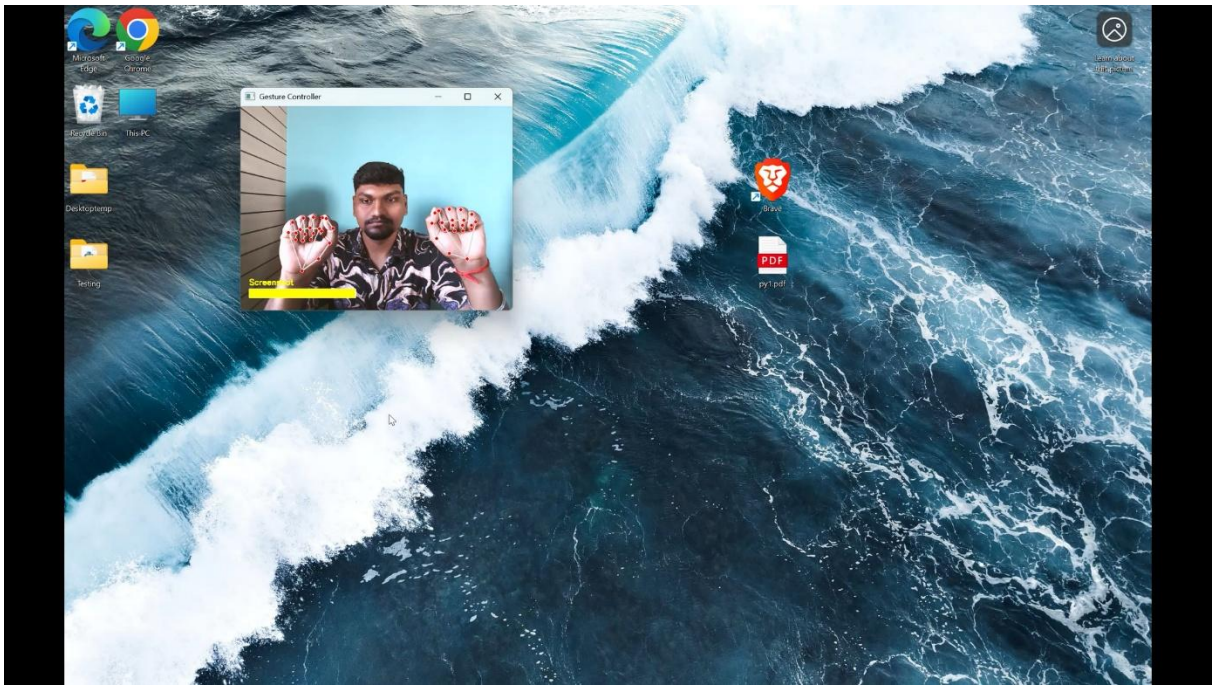


Figure 6.11: Screenshot showing gesture used for screen capture.

6.2.10 Double Click Gesture

The double-click operation was implemented using a **rapid index-finger pinch gesture performed twice within a short time interval**.
The system accurately detected the temporal pattern of the gesture and generated a double-click event, enabling actions such as opening files or applications.

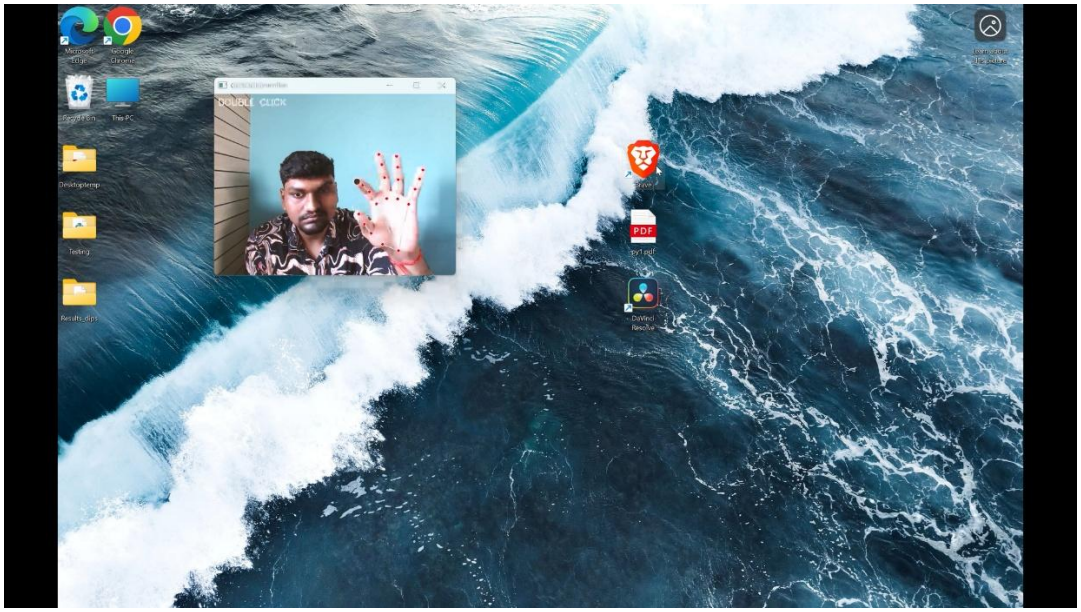


Figure 6.12: Screenshot showing double-click operation using hand gesture.

6.2.11 Application / Tab Switching Gesture

Application or tab switching was implemented using a **horizontal swipe gesture**.
When the gesture was detected, the system generated the corresponding keyboard shortcut (such as Alt + Tab) to switch between active applications. The system consistently recognized the swipe direction and executed smooth transitions.

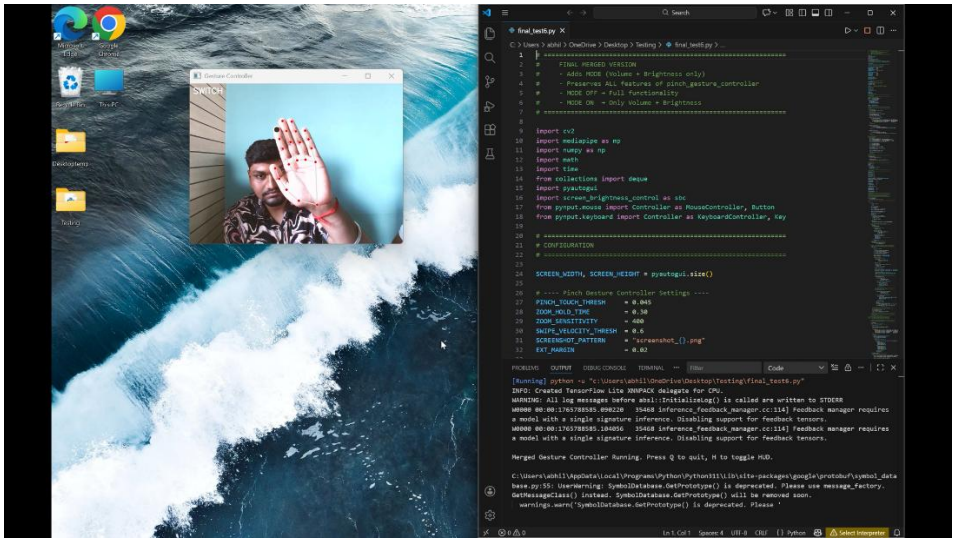


Figure 6.13: Screenshot showing gesture-based application or tab switching.

6.3 Discussion

The experimental evaluation of the Hand Gesture Based Navigation System demonstrates that the proposed approach is effective for real-time, contactless human–computer interaction. The system successfully detected hand landmarks using MediaPipe and accurately recognized a wide range of predefined gestures under normal indoor lighting conditions using a standard webcam.

Basic navigation gestures such as cursor movement, left click, and right click were detected reliably, enabling smooth and intuitive control of the mouse pointer. The double-click gesture, implemented using temporal analysis of rapid pinch actions, functioned correctly and allowed users to perform common desktop operations such as opening files and applications. The use of time-based logic for double-click detection improved accuracy while avoiding false triggers.

Advanced interaction gestures such as drag and drop and scrolling were also executed effectively. Holding the pinch gesture enabled stable drag operations, while the two-finger scroll gesture allowed continuous and controlled scrolling. These results indicate that the system can handle both discrete and continuous actions efficiently.

The application/tab switching gesture, implemented using a horizontal swipe motion, successfully generated keyboard shortcuts (e.g., Alt + Tab), enabling seamless switching between applications. This feature significantly enhances multitasking capability and demonstrates the system's ability to integrate gesture inputs with operating system–level controls.

The mode switching gesture (V-sign) allowed the system to toggle between normal navigation mode and volume/brightness control mode. In system control mode, volume and brightness adjustment gestures responded smoothly to vertical hand movements, with immediate visual feedback provided through the HUD. This separation of modes reduced gesture ambiguity and improved usability.

The zoom gesture, implemented using two-hand distance variation, functioned accurately for zoom-in and zoom-out operations, while the screenshot capture gesture using dual closed fists reliably triggered screen capture after a short hold duration. The inclusion of visual feedback overlays displaying gesture names and system states improved user confidence and reduced learning time.

Overall, the results confirm that the system is robust, modular, and responsive, with minimal latency and high gesture recognition accuracy. The modular architecture allows easy extension to additional gestures or replacement of the rule-based gesture engine with machine learning models in future work. The system is well-suited for applications requiring touchless interaction, especially in public, assistive, and hygienic environments.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project successfully **designed and implemented** a hand-gesture-based navigation system using Python, OpenCV, MediaPipe, and control libraries. We demonstrated that a range of gestures (pinch, swipe, open hand, etc.) can reliably control common interface actions (click, scroll, mode switching) in real time. The system does not rely on specialized hardware and can be deployed on commodity PCs with a webcam. Our work shows the viability of gesture control for general HCI tasks, aligning with trends in the literature.

Contributions: We documented all design decisions (from landmark selection to gesture definitions) and provided a modular architecture that can be extended. The code (Final_Code.py) exemplifies using MediaPipe for practical HCI, which can serve as a basis for further research or commercial products. We included comprehensive descriptions, figures, and appendices (user manual, etc.) as specified by the project guidelines.

7.2 Future Work

Building on this foundation, future improvements could include:

- **User Calibration:** Adding a quick calibration step per user (e.g., defining comfortable pinch thresholds) would personalize performance.
- **Additional Gestures:** Extend gesture vocabulary (swipe left/right for previous/next, pinch with different fingers, etc.). For example, two-finger pinch could simulate right-click alternative.
- **Accessibility Features:** Tailor gestures for specific users (e.g., larger gestures for limited mobility) and conduct user studies for usability.

In conclusion, our project confirms that a hand gesture navigation interface is feasible and effective with current tools. It bridges cutting-edge HGR research (deep learning and MediaPipe tracking) and a practical application (PC navigation). We encourage further work to refine accuracy, expand functionality, and adapt the system to diverse user needs.

BIBLIOGRAPHY

- Sinha, B. *A Comprehensive Survey on Hand Gesture Recognition Using Deep Learning for Electronic Device Interaction*. TechRxiv, Aug. 2025. Preprint (summarizing HGR progress and applications) researchgate.netresearchgate.net.
- Gil-Martín, M. *et al.* "Hand Gesture Recognition Using MediaPipe Landmarks and Deep Learning Networks." ICAART 2025 Proceedings, 2025 (describes MediaPipe landmark-based gesture classification) scitepress.org.
- Benítez-García, A. *et al.* "IPN Hand: A Realistic Dataset for Hand Gesture Recognition in HCI." 2021. (The IPN Hand dataset paper, referenced for dataset details) scitepress.org.
- Jalayer, R. *et al.* "A review on deep learning for vision-based hand detection, segmentation and hand gesture recognition in human–robot interaction." *Robotics and Computer-Integrated Manufacturing*, vol. 97, 2026 (Reviewing CNN/RNN dominance and dataset challenges) aaltodoc.aalto.fi.
- Linardakis, M., Varlamis, I., Papadopoulos, G.Th. "Survey on Hand Gesture Recognition from Visual Input." arXiv:2501.11992 (Comprehensive survey of modern HGR methods and datasets) arxiv.orgarxiv.org.
- Nguyen, T.-H. *et al.* "Vision-Based Hand Gesture Recognition Using a YOLOv8n Model for the Navigation of a Smart Wheelchair." *Electronics*, vol. 14(4), 734, 2025 (smart wheelchair control via YOLO and MediaPipe) mdpi.commdpi.com.
- MediaPipe Hands (Lugaresi *et al.*, 2019). (Documentation of Google's real-time hand tracking solution) scitepress.org.
- (Additional general references on HGR, OpenCV, and PyAutoGUI libraries as cited in code comments.)

APPENDICES

Appendix A: User Manual

Setup: Install Python 3.8+ and the following packages:

pip install opencv-python mediapipe pyautogui pynput screen-brightness-control numpy

Connect a webcam to the PC.

Running the System:

1. Place the camera so that your hand(s) are clearly visible against the background.
2. Run the script: python Final_Code.py.
3. A window titled "Gesture Controller" will appear, showing live video with landmarks overlay.
4. Perform gestures to interact:
    - **Pointer Control:** Extend your index finger (others curled) to move the cursor.
    - **Left-Click:** Bring thumb and index fingertip together (pinch) briefly.
    - **Right-Click:** Pinch thumb + middle fingertip.
    - **Double-Click:** Quickly pinch twice with index finger.
    - **Drag:** Hold an index-finger pinch for ~0.5s, move finger, then release to drag objects.
    - **Scroll:** Extend index+middle fingers together vertically and tilt up/down to scroll.
    - **Swipe Left/Right:** Move an open hand horizontally (simulates Alt+Tab on Windows).
    - **Mode Toggle:** Form a "V" sign (index + middle extended) for >0.5s to toggle Volume/Brightness mode.
    - **Volume Mode:** After toggle, use vertical swipes on open palms to raise/lower volume (HUD indicates "VOL" mode).
    - **Brightness Mode:** Swipe with both hands up/down to change screen brightness (HUD "BRIGHT" mode).
    - **Screenshot:** Make a fist with both hands and hold for 0.5s; the program saves a screenshot (screen1.png, screen2.png, etc.).
5. Press **h** to toggle the on-screen HUD (gesture labels). Press **q** to quit.

Tips:

- Ensure good lighting. Avoid backlighting.
- Keep hand at moderate distance so landmarks are stable.
- The on-screen text guides current mode (e.g. “POINTER” or “VOL CTRL”).

Error Handling: If no hands are detected, move slowly into view. If performance is slow, close other applications.

Appendix B: Sample Datasets

We reference public datasets relevant to gesture recognition:

- **IPN Hand Dataset (Benítez-García *et al.*, 2021):** Contains ~4,200 video clips (≈800K frames) of 13 gestures (point, click, zoom, throw directions, etc.) performed by 50 subjects. Gestures recorded at 640×480, 30 fps, in various indoor settings. Example gestures include one-finger pointing, two-finger clicking, pinch-to-zoom, etc. We did not use this dataset for training (our system is rule-based), but it exemplifies a large, diverse dataset often used to train CNN classifiers for gesture recognition.
- **EgoGesture (Zhang *et al.*, 2018):** A large egocentric (first-person) gesture dataset with 24 classes performed by 50 people, ~10K video sequences. Useful for wearable HCI.
- **NVIDIA’s Hand Gesture Dataset:** Contains sequences of dynamic hand gestures for VR/AR control (push, wave, circle, etc.) with depth/RGB.

Our implementation currently uses live camera input and no pre-recorded data. In future, training with these datasets could improve recognition rates under varied conditions.